



Web Storage

چیست؟



farnoodReact



farnoodlotfali



Web Storage یک API است که راهی را فراهم می‌کند تا مرورگرها بتوانند جفت‌های key/value را به صورت locally در مرورگر کاربر ذخیره کنند.

دو روش برای ذخیره داده ارائه می‌دهد:

sessionStorage

localStorage

localStorage

No expiration date
Same across tabs

sessionStorage

Cleared on tab close
Per tab only

هر دو داده را به صورت key/value ذخیره می‌کنند

Local Storage در برابر Session Storage

هر دو بخشی از Web Storage API هستند و متدهای یکسانی دارند، اما در مدت زمان ماندگاری داده تفاوت دارند.

Session Storage



داده‌ها را برای یک session ذخیره می‌کند؛ با بسته شدن تب از بین می‌رود.

- داده فقط برای session همان تب است
- با بسته شدن تب به‌طور خودکار پاک می‌شود
- بین تب‌های مختلف مشترک نیست

Local Storage



داده‌ها را برای origin فعلی و بدون تاریخ انقضا ذخیره می‌کند.

- داده تا حذف صریح باقی می‌ماند
- پس از بسته شدن مرورگر هم می‌ماند
- بین همه تب‌های یک origin مشترک است

مفاهیم کلیدی



بلوک‌های اصلی که با Web Storage API با آن‌ها کار خواهید کرد



Storage Object

هر origin فضای ذخیره‌سازی مجزای خودش را دارد.

```
window.localStorage
```



sessionStorage

فقط برای session همان تب زنده است.

```
sessionStorage.setItem(...)
```



localStorage

بدون تاریخ انقضا باقی می‌ماند، تا زمانی که حذف شود.

```
localStorage.setItem(...)
```



Key / Value

داده به صورت جفت‌های رشته‌ای ساده مثل یک کلید و مقدار آن ذخیره می‌شود.

```
setItem(key, value)
```



مثال: localStorage

localStorage داده را برای origin فعلی و بدون تاریخ انقضا نگه می‌دارد، حتی پس از بسته شدن مرورگر.



remove.js

```
// Remove one item
localStorage.removeItem("username");

// Remove everything
localStorage.clear();

// Still there after refresh / restart
// until explicitly removed
```



save.js

```
// Save data
localStorage.setItem("username", "farnood");

// Read data
const user = localStorage.getItem("username");
console.log(user); // "farnood"
```



localStorage داده‌ها پس از رفرش صفحه و حتی بسته شدن مرورگر باقی می‌مانند.



مثال: sessionStorage

sessionStorage به همان شکل کار می‌کند، اما با بسته شدن همان تب مرورگر، داده از بین می‌رود.



lifecycle.js

```
// Opening a new tab to the same site  
// does NOT share this data
```

```
// Closing this tab clears it  
sessionStorage.removeItem("step");  
sessionStorage.clear();
```



save.js

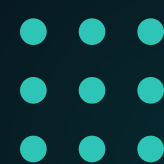
```
// Save data for this tab only  
sessionStorage.setItem("step", "2");
```

```
// Read data  
const step = sessionStorage.getItem("step");  
console.log(step); // "2"
```



sessionStorage برای داده‌ای که نباید از یک session تب بیشتر عمر کند، عالی است.

i



Web Storage در برابر Cookies

ویژگی	Cookies	Web Storage
حجم ذخیره سازی	حدود ۴ کیلوبایت	۵ مگابایت یا بیشتر
ارسال به سرور	بله، با هر درخواست	خیر، در مرورگر می ماند
انقضا	به صورت دستی تنظیم می شود	localStorage: ندارد / sessionStorage: با بسته شدن تب
نوع API	رشته ای، نیاز به parse دستی	ساده: getItem / setItem
دسترسی	کلاینت و سرور	فقط مرورگر (کلاینت)

وقتی سرور هم به داده نیاز دارد از Cookies استفاده کنید؛ برای داده مخصوص مرورگر از Web Storage.

نکات کلیدی



Browser-based Key/Value Storage

Web Storage جفت‌های کلید/مقدار را به صورت محلی در مرورگر کاربر ذخیره می‌کند.

۱

More Intuitive than Cookies

متدهای ساده `setItem` / `getItem`، بدون نیاز به `parse` دستی.

۲

localStorage

داده را برای `origin` فعلی و بدون تاریخ انقضا ذخیره می‌کند.

۳

sessionStorage

داده را برای یک `session` ذخیره می‌کند؛ با بسته شدن تب از بین می‌رود.

۴

